

A database-management system (DBMS):

1. is a **collection** of *interrelated data* and a *set of programs* to access those data. The collection of data, usually referred to as the database, contains information relevant to an project/organization/task.
2. The primary **goal** of a DBMS is *to provide a way to store and retrieve database information* that is both convenient and efficient. Database systems are designed to manage large bodies of information.
3. **Management of data involves** both *defining structures* for storage of information and *providing mechanisms for the manipulation of information*.
4. In addition, the database system must **ensure** *the safety of the information stored*, despite system crashes or attempts at unauthorized access.
5. **If data are to be shared** among several users, the *system must avoid possible anomalous* (deviating from what is expected) results. Because information is so important in most organizations, computer scientists have developed a large body of concepts and techniques for managing data.

Database System Applications

Databases are widely used. Here are some representative applications:

- **Enterprise Information:**

- **Sales:** For customer, product, and purchase information.
- **Accounting:** For payments, receipts, account balances, assets and other accounting information.
- **Human resources:** For information about employees, salaries, payroll taxes, and benefits, and for generation of paychecks.
- **Manufacturing:** For management of the supply chain and for tracking production of items in factories, inventories of items in warehouses and stores, and orders for items.

◦ **Online retailers:** For sales data noted above plus online order tracking, generation of recommendation lists, and maintenance of online product evaluations.

- **Banking and Finance:**

◦ **Banking:** For customer information, accounts, loans, and banking transactions.

◦ **Credit card transactions:** For purchases on credit cards and generation of monthly statements.

◦ **Finance:** For storing information about holdings, sales, and purchases of financial instruments such as stocks and bonds; also for storing real-time market data to enable online trading by customers and automated trading by the firm.

- **Universities:** For student information, course registrations, and grades (in addition to standard enterprise information such as human resources and accounting).

- **Airlines:** For reservations and schedule information. Airlines were among the first to use databases in a geographically distributed manner.

- **Telecommunication:** For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards, and storing information about the communication networks.

Purpose of Database Systems

Database systems arose in response to early methods of computerized management of commercial data.

As an **example**,

1. consider part of a **university organization** that, among other data, **keeps information about all instructors, students, departments, and course offerings**.
2. **One way to keep the information on a computer** is to store it in operating system files.
3. To **allow users to manipulate the information**, the system has a number of application programs that manipulate the files, including programs to:

- **Add** new students, instructors, and courses
- **Register** students for courses and generate class rosters
- **Assign** grades to students, compute grade point averages (GPA), and generate transcripts

This typical **file-processing system** is supported by a conventional operating system.

The **system stores permanent records in various files**, and it **needs different application programs to extract records** from, and **add** records to, the appropriate **files**. **Before database management systems (DBMSs) were introduced**, organizations usually stored information in such systems.

Keeping organizational information in a **file-processing system** has a number of major **disadvantages**:

1. **Data redundancy and inconsistency**. Since **different programmers create the files** and application programs over a long period, the **various files** are likely to have **different structures** and the programs may be written in several programming languages.
2. **Difficulty in accessing data**. Suppose that one of the university **clerks needs to find out the names of all students who live within a particular postal-code area**. The clerk asks the data-processing department to generate such a list. Because the designers of the original system did not anticipate this request, there is **no application program on hand to meet it**.
3. **Data isolation**. Because **data are scattered in various files**, and files may be in **different formats**
4. **Integrity problems**. The data values stored in the database must satisfy certain types of **consistency constraints**.
5. **Atomicity problems**. A computer system, like any other **device**, is **subject to failure**. Atomicity problems **occur when a transaction in a Database Management System is not executed completely** or **not rolled back entirely** in case of failure, leading to partial updates that can cause data inconsistency.

6. **Concurrent-access anomalies.** For the sake of overall performance of the system and faster response, many systems allow multiple users to update the data simultaneously.
7. **Security problems.** Not every user of the database system should be able to access all the data. For example, in a university, payroll personnel need to see only that part of the database that has financial information.

View of Data

A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data.

A major purpose of a database system is to provide users with an *abstract view of the data*. That is, the system hides certain details of how the data are stored and maintained.

Data Abstraction

For the system to be usable, it must retrieve data efficiently.

The need for efficiency has let designers to use complex data structures to represent data in the database.

Since many database-system users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interactions with the system:

- **Physical level.** The lowest level of abstraction describes *how* the data are actually stored. The physical level describes complex low-level data structures in **detail**.

Example:

When we access data we may get a single data or a table of data. Moreover, by the term "relational database" we visualize a table of rows and columns. But at a physical level, these tables are stored in hard drives which are located at a very secure data center.



SCALER
Topics

Above is the picture of a Google data center that can be visited by only 1% of Googlers! All these racks contain hard disk drives storing all your secured data!

- **Logical level.** The next-higher level of abstraction describes what data are stored in the database, and what relationships exist among those data. The logical level thus describes the entire database in terms of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. This is referred to as physical data independence. **Database administrators**, who must decide what information to keep in the database, use the logical level of abstraction.

Example:

We have data of a few products like product id, product name, and manufacturing date, and we have another set of data of customers containing customer id, customer name, and customer address. Now, we need to frame this data in proper tables of products and customers. After that, we can even frame a join to show which product has been ordered by which customer.

- **View level.** The highest level of abstraction describes only part of the entire database. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database.

Many users of the database system do not need all this information; instead, they need to access only a part of the database.

The view level of abstraction exists to simplify their interaction with the system.

Example:

Concerning the example in the logical level section, let us say a **customer** wants to view the **order** history, he gets to see only the orders he had made in the **past**. Now, let us say a shop owner needs to see the products that are on the **order list**. He gets to see a table containing all the info about the products and the customers to whom they need to be delivered.

The system may provide many views for the same database. Figure 1.1 shows the relationship among the three levels of abstraction.

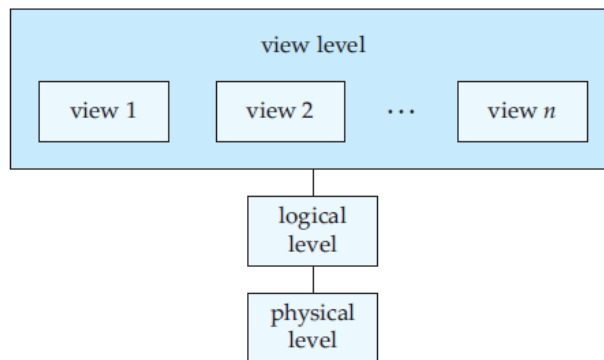


Figure 1.1 The three levels of data abstraction.

languages support the notion of a structured type. For example, we may describe a record as follows:

type *instructor* = **record**

ID : **char** (5);

name : **char** (20);

dept name : **char** (20);

salary : **numeric** (8,2);

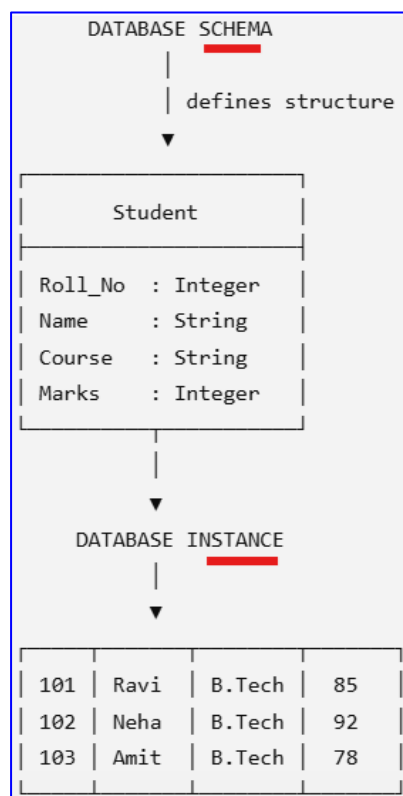
end;

This code defines a new record type called *instructor* with four fields. Each field has a name and a type associated with it. A **university organization** may have several such record types, including

- *department*, with fields *dept name*, *building*, and *budget*
- *course*, with fields *course id*, *title*, *dept name*, and *credits*
- *student*, with fields *ID*, *name*, *dept name*, and *tot cred*

At the physical level, an *instructor*, *department*, or *student* record can be described as a block of consecutive storage locations. The compiler hides this level of detail from programmers.

Instances and Schemas:



Databases change over time as information is inserted and deleted.

The collection of information stored in the database at a particular moment is called an instance of the database.

The overall design of the database is called the database schema. Schemas are changed infrequently, if at all.

The concept of database schemas and instances can be understood by analogy to a program written in a programming language.

A database **schema** corresponds to the **variable** declarations (along with associated type definitions) in a program.

Schema	Instance
Structure/design of database	Actual data in database
Usually changes infrequently	Changes frequently
Describes tables, columns, data types, relationships	Contains actual records
Like a blueprint	Like the current building
Example: Roll_No INT, Name VARCHAR	Example: 101 , Ravi

Each **variable** has a particular value at a given **instant**. The values of the variables in a program at a point in time correspond to an **instance** of a database schema.

Database systems have several schemas, partitioned according to the levels of abstraction. The **physical schema** describes the database design at the **physical level**, while the **logical schema** describes the database design at the **logical level**. A database may also have several schemas at the **view level**, sometimes called **subschemas**, that describe different views of the database.

Data Models

Underlying the structure of a database is the **data model**: a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.

A data model provides a way to describe the design of a database at the physical, logical, and view levels.

The data models can be classified into **four** different categories:

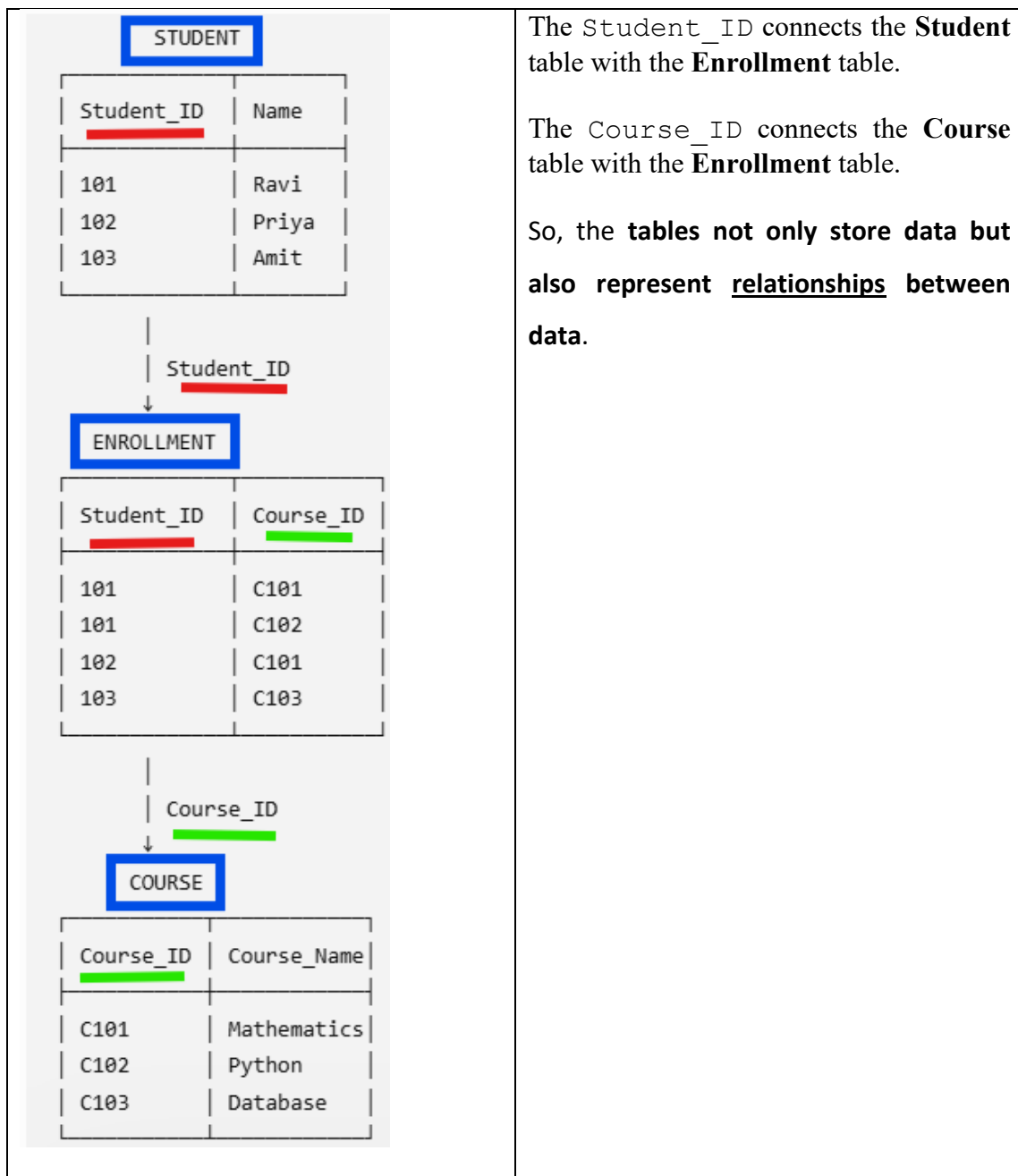
- **Relational Model**:

The relational model uses a **collection** of tables to represent both **data** and the **relationships** among those data. Each table has multiple columns, and each column has a unique name. Tables are also known as **relations**. The relational model is an example of a **record-based model**. The relational data model is the most widely used data model, and a vast majority of current database systems are based on the relational model.

The **Relational Model** is a way of storing data in the form of **tables**.

A table consists of:

- **Rows** → individual records
- **Columns** → properties/attributes of the records
- **Table** → called a **relation**



The Student_ID connects the **Student** table with the **Enrollment** table.

The Course_ID connects the **Course** table with the **Enrollment** table.

So, the **tables not only store data but also represent relationships between data.**

- **Entity-Relationship Model:**

The entity-relationship (E-R) data model uses a [collection of basic objects, called *entities*, and *relationships* among these objects](#). An entity is a “thing” or “object” in the real world that is distinguishable from other objects. The entity-relationship model is widely used in database design.

Here:

- **Student** → Entity
- **Course** → Entity
- **Enrolls in** → Relationship

Simple E-R Diagram

```
graph TD
    Student[STUDENT] -- ENROLLS IN --> Course[COURSE]
```

This diagram tells us:

A Student enrolls in a Course.

An **attribute** describes an entity.

For example, a student may have:

Student

- |
- |— Student_ID
- |— Name
- |— Email
- |— Semester

These are the **attributes** of the Student entity.

- **Object-Based Data Model:**

Object-oriented programming (especially in [Java](#), [C++](#), or [C#](#)) has become the dominant software-development methodology. This led to the development of an object-oriented data model that can be seen as [extending the E-R model with notions](#)

of [encapsulation, methods \(functions\), and object identity](#). The object-relational data model [combines](#) features of the object-oriented data model and relational data model.

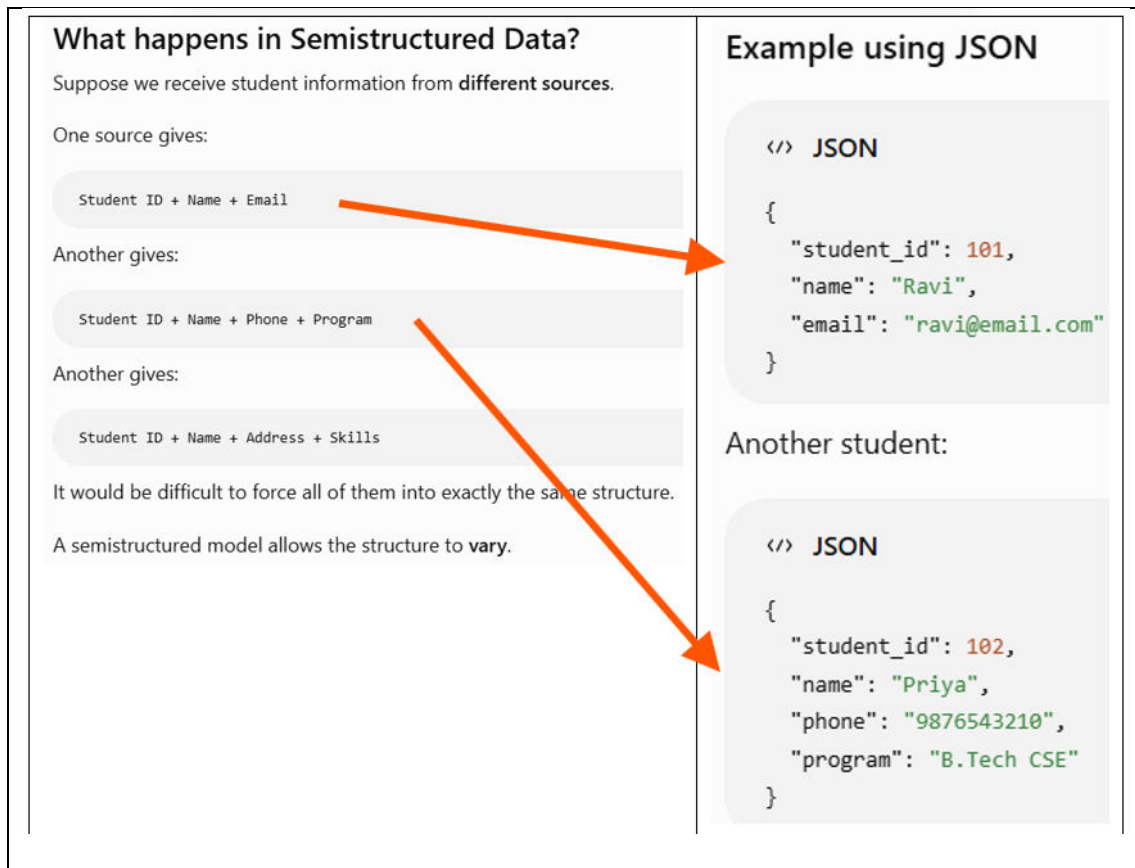
Suppose we are designing a university system. A <u>Student</u> can be represented as an object: STUDENT Data / Attributes Student_ID = 101 Name = "Ravi" Program = "B.Tech CSE" Methods / Functions calculateGrade() displayDetails() registerCourse() So the object contains both information and operations.	Every object has a unique identity that distinguishes it from other objects. For example: Student Object 1 ID = 101 Name = Ravi Student Object 2 ID = 102 Name = Priya
--	---

- **Semistructured Data Model:**

The **semistructured data model** is a way of representing and managing data where the structure is **not fixed or rigid** like in traditional relational databases, but is still organized enough to allow meaningful interpretation.

- **Semistructured data** does not follow a strict schema like relational databases.
 1. The structure can vary from record to record, and the same data field might be missing or have different types in different records.
 2. It is often represented in formats such as **XML, JSON**

The **Semistructured Data Model** is a data model used in DBMS to represent data when the data **does not have a fixed structure**.



Key Characteristics

1. **Flexible schema** – No predefined table structure; schema can evolve with data.
2. **Self-describing** – Data contains information about its own structure (e.g., **tags** in XML, **keys** in JSON).
3. **Irregular structure** – **Fields** may differ between records; hierarchy and nesting are common.
4. **Easier integration** – Can accommodate data from multiple sources with different structures.
5. **Human-readable formats** – Often stored in text-based representations.

Database Languages

A database system provides a **data-definition language** to specify the database schema and a **data-manipulation language** to express database queries and updates.

1. Data-Manipulation Language

A **data-manipulation language (DML)** is a language that enables users to access or manipulate data as organized by the appropriate data model. The types of access are:

- Retrieval of information stored in the database

- Insertion of new information into the database
- Deletion of information from the database
- Modification of information stored in the database

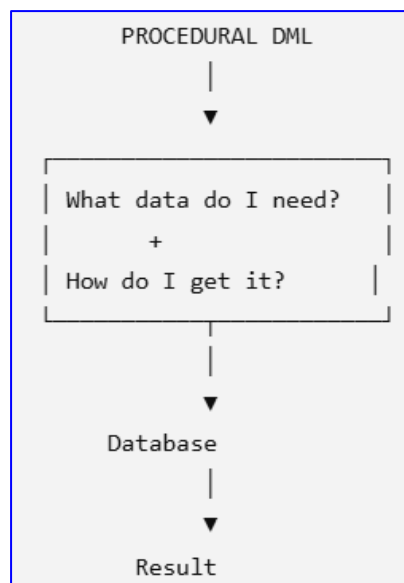
For example:

- Retrieve data → SELECT
- Add data → INSERT
- Modify data → UPDATE
- Delete data → DELETE

There are basically **two** types of **DML**:

I. Procedural DMLs: require a user to specify *what* data are needed and *how* to get those data. (e.g. **PL/SQL**)

What data I want + How to get it



II. Declarative DMLs: (also referred to as **nonprocedural DMLs**) require a user to specify *what* data are needed **without** specifying *how* to get those data.

(e.g. **SQL** query)

In **Declarative DML**, you tell the database:

What data I want

You do not tell it how to find the data.

A **query** is a statement requesting the retrieval of information. The portion of a DML that involves information retrieval is called a **query language**.

Example:

```
SELECT Name
FROM Student
WHERE Marks > 80;
```

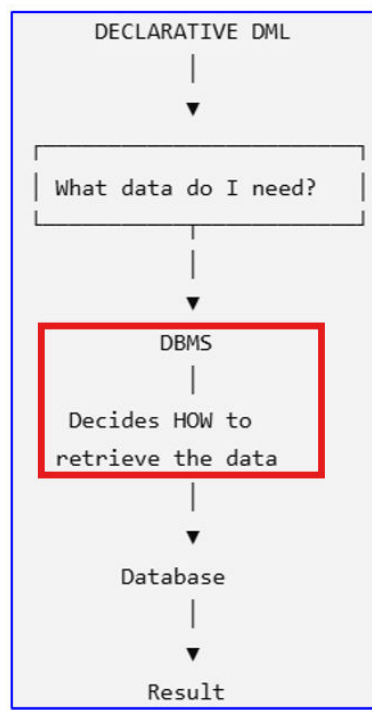
We specify:

What do we want?
Names of students whose marks are greater than 80.

We don't specify:

"First scan row 1, then row 2, then row 3..."

The **DBMS** determines how to execute the query efficiently.



2. Data-Definition Language

We specify a database schema by a set of definitions expressed by a special language called a **data-definition language (DDL)**. The DDL is also used to specify additional properties of the data.

DDL tells the DBMS what the database should look like.

For example, suppose we want a `Student` table.

We can define it using SQL:

Example-1 <pre></> SQL CREATE TABLE Student (Roll_No INT, Name VARCHAR(50), Marks INT);</pre> This tells the DBMS: <pre>Create a table called Student --- Roll_No → Integer --- Name → Character/String --- Marks → Integer</pre>	Example-2 DDL can define these tables and their relationships.
---	---

We specify the **storage** structure and **access methods** used by the database system by a set of statements in a special type of DDL called a **data storage and definition language**.

These statements define the implementation details of the database schemas, which are usually hidden from the users (**data abstraction**).

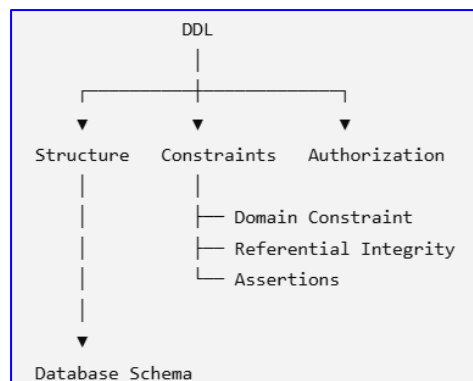
The data values stored in the database must satisfy certain consistency constraints.

- **Domain Constraints**. A domain of possible values must be associated with every attribute (for **example**, **integer** types, **character** types, **date** types).

DDL is not only about creating tables.

It can also specify **rules that the data must follow**.

These rules are called **constraints**.



Example of domain constraint:

Domain constraint specifies what type of values are allowed in a column.

For example:

```

SQL
CREATE TABLE Student (
  Roll_No INT,
  Name VARCHAR(50),
  Date_of_Birth DATE,
  Marks INT
);
  
```

- **Referential Integrity**. There are cases where we wish to ensure that a **value that appears in one relation for a given set of attributes also appears in a certain set of attributes in another relation (referential integrity)**. For **example**, the **dept name** value in a **course** record must appear in the **dept name** attribute of some record of the **department** relation. Database modifications can cause violations of referential integrity.

Dept_Name
Computer Science
Mathematics
Physics

Course_ID	Course_Name	Dept_Name
C101	DBMS	Computer Science
C102	Calculus	Mathematics
P101	Mechanics	Physics

Now suppose someone tries to insert:

Course_ID	Course_Name	Dept_Name
C103	AI	Chemistry

But Chemistry does not exist in the Department table

Referential Integrity = A reference to another table must point to an existing record.

- **Assertions.** An assertion is any condition that the database must always satisfy. Domain constraints and referential-integrity constraints are special forms of assertions. However, there are many constraints that we cannot express by using only these special forms. For **example**, “**Every department must have at least five courses offered every semester**” must be expressed as an assertion.

An **assertion** is a condition that the database must always satisfy.

It is a more general type of constraint.

Consider this rule:

Every department must offer at least five courses every semester.

This rule is not simply about the data type of one column.

Department

↓

Computer Science

DBMS

Operating Systems

Computer Networks

AI

Machine Learning

↓

5 courses ✓

- **Authorization.** We may want to differentiate among the users as far as the type of access they are permitted on various data values in the database. These differentiations are expressed in terms of authorization, the most common being: **read authorization**, which allows reading, but **not modification**, of data; **insert authorization**, which allows insertion of new data, but **not modification** of existing data; **update authorization**, which allows modification, but **not deletion**, of data; and **delete authorization**, which allows deletion of data. We may assign the user *all, none, or a combination of these types of authorization*.

Now suppose our university database has different users.

For example:

UNIVERSITY DATABASE

↓

Teacher

Student

Admin

They should not all have the same permissions.

Example:

User	Read	Insert	Update	Delete
Student	✓	✗	✗	✗
Teacher	✓	✓	✓	✗
Admin	✓	✓	✓	✓

Relational Databases

A relational database is based on the relational model and uses a collection of tables to represent both data and the relationships among those data. It also includes a DML and DDL.

Tables

Each table has multiple columns and each column has a unique name. Figure 1.2 presents a sample relational database comprising two tables: one shows details of university instructors and the other shows details of the various university departments.

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The instructor table

dept_name	building	budget
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The department table

1.5.2 Data-Manipulation Language

The SQL query language is nonprocedural. A query takes as input from several tables (possibly only one) and always returns a single table. Here is an example of an SQL query that finds the names of all instructors in the **History** department:

```
select instructor.name  
  
from instructor  
  
where instructor.dept name = 'History';
```

```
select instructor.ID, department.dept name  
  
from instructor, department  
  
where instructor.dept name = department.dept name and  
department.budget > 95000;
```

Data-Definition Language

SQL provides a rich DDL that allows one to define tables, integrity constraints, assertions, etc.

For instance, the following SQL DDL statement defines the *department* table:

```
create table department  
  
(dept name char (20), building char (15), budget numeric (12,2));
```

CHAR → fixed size

VARCHAR2 → variable size and recommended in Oracle

VARCHAR → variable size, but avoid using it in Oracle

numeric (5,2)... > 5 means max five digit including decimal... 2 means max two digit in decimal

Example :

100.12 --> **ok**

Database Access from Application Programs

SQL does not support actions such as input from users, output to displays, or communication over the network. Such computations and actions must be written in a *host* language, such as C, C++, or Java, with embedded SQL queries that access the data in the database. Application programs are programs that are used to interact with the database in this fashion.

Example:

By providing an application program interface (set of procedures) that can be used to send DML and DDL statements to the database and retrieve the results.

The Open Database Connectivity (ODBC) standard for use with the C language is a commonly used application program interface standard.

The Java Database Connectivity (JDBC) standard provides corresponding features to the Java language.

Database Design

Database design mainly involves the design of the database schema.

1.6.1 Design Process

A high-level data model provides the database designer with a conceptual framework in which to specify the data requirements of the database users, and how the database will be structured to fulfill these requirements.

The outcome of this phase is a specification of user requirements.

Next, the designer chooses a data model, and by applying the concepts of the chosen data model, translates these requirements into a conceptual schema of the database.

The schema developed at this conceptual-design phase provides a detailed overview.

In terms of the **relational model**, the conceptual-design process involves decisions on what attributes we want to capture in the database and how to group these attributes to form the various tables.

A fully developed conceptual schema **indicates the functional requirements** of the enterprise. In a **specification of functional requirements**, users describe the kinds of operations (or transactions) that will be performed on the data.

The **process of moving from an abstract data model to the implementation** of the database proceeds in **two** final design phases. In the **logical-design phase**, the designer maps the high-level conceptual schema onto the implementation data model. In the subsequent **physical-design phase**, in which include the form of **file** organization and the internal storage structures.

Example:

Database Design for a University Organization

The initial specification of user requirements may be based on **interviews** with the database users, and on the **designer's own analysis of the organization**.

The description that arises from this design phase serves as the basis for specifying the **conceptual structure of the database**.

Here are the major characteristics of the university.

- The university is **organized into departments**. Each department is identified by a unique name (*dept name*), is located in a particular *building*, and has a *budget*.
- Each department has a list of **courses** it offers. Each course has associated with it a *course id*, *title*, *dept name*, and *credits*, and may also have associated *prerequisites*.
- **Instructors** are identified by their unique *ID*. Each instructor has *name*, associated department (*dept name*), and *salary*.

- **Students** are identified by their unique *ID*. Each student has a *name*, an associated major department (*dept name*), and *tot cred* (total *credit* hours the student earned thus far).
- The university maintains a list of **classrooms**, specifying the name of the *building*, *room number*, and room *capacity*.
- The university maintains a **list of all classes (sections)** taught. Each section is identified by a *course id*, *sec id*, *year*, and *semester*, and has associated with it a *semester*, *year*, *building*, *room number*, and *time slot id* (the time slot when the class meets).
- The department has a **list of teaching assignments** specifying, *for each instructor*, the sections the instructor is teaching.
- The university has a **list of all student course registrations**, specifying, for each *student*, the courses and the associated sections that the student has taken (registered for).

This simplified model to help you understand conceptual ideas of data base design.

The Entity-Relationship Model

The entity-relationship (E-R) data model uses a collection of basic objects, called *entities*, and *relationships* among these objects. An *entity* is a “thing” or “object” in the *real* world that is distinguishable from other objects. For example, *each person is an entity*, and *bank accounts can be considered as entities*.

Entities are described in a database by a set of *attributes*. For example, the attributes *dept name*, *building*, and *budget* may describe one particular *department* in a university, and they form attributes of the *department* entity set. Similarly, attributes *ID*, *name*, and *salary* may describe an *instructor* entity.

The extra attribute *ID* is used to identify an instructor uniquely (since it may be possible to have two instructors with the same name and the same salary).

A unique instructor identifier must be assigned to each instructor.

A **relationship** is an association among several entities. For example, a **member** relationship associates an instructor with her department. The set of all entities of the same type and the set of all relationships of the same type are termed an **entity set** and **relationship set**, respectively.

The overall logical structure (schema) of a database can be expressed graphically by an *entity-relationship (E-R) diagram*.

There are several ways in which to

draw these diagrams. One of the most popular is to use the **Unified Modeling Language (UML)**.

In the notation we use, which is based on UML, an E-R diagram is represented as follows:

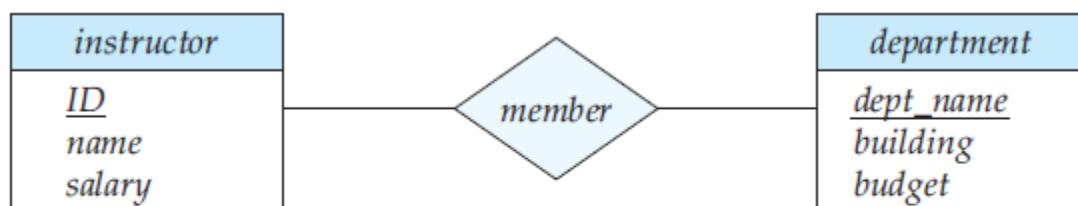


Figure 1.3 A sample E-R diagram.

- **Entity** sets are represented by a **rectangular** box with the entity set name in the header and the attributes listed below it.
- **Relationship** sets are represented by a **diamond** connecting a pair of related entity sets. The name of the relationship is placed inside the diamond.

The E-R diagram indicates that there are two entity sets, *instructor* and *department*, with attributes as outlined earlier. The diagram also shows a relationship *member* between *instructor* and *department*.

One important constraint is **mapping cardinalities**, which express the number of entities to which another entity can be associated via a relationship set. For example, if each **instructor must be associated with only a single department**, the E-R model can express that constraint.

Normalization

Another method for designing a relational database is to use a process commonly known as normalization. The goal is to generate a set of relation schemas that allow us to store information without unnecessary redundancy and retrieve information easily. The approach is to design schemas that are in an appropriate normal form. To determine whether a relation schema is in one of the desirable normal forms, we need additional information about the real-world enterprise that we are modelling with the database. The most common approach is to use functional dependencies.

To understand the need for normalization, let us look at what can go wrong in a bad database design. Among the undesirable properties that a bad design may have are:

- Repetition of information
- Inability to represent certain information

Database Architecture

Below picture depicts the various components of a database system and the connections among them.

The architecture of a database system is greatly influenced by the underlying computer system on which the database system runs.

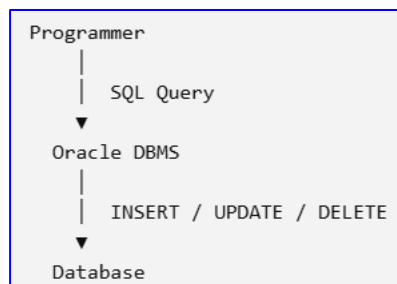
Database systems can be centralized, or client-server, where one server machine executes work on behalf of multiple client machines.

Distributed databases span multiple geographically separated machines.

Most users of a database system today are connected to it through a network. We can, therefore, differentiate between client machines, on which remote database users work, and server machines, on which the database system runs.

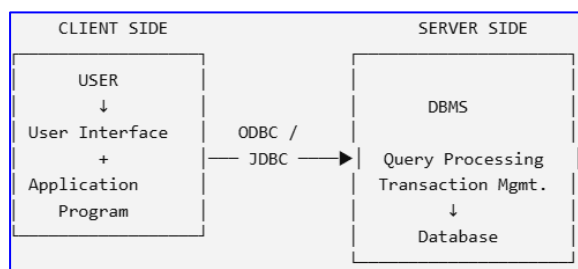
1-Tier Architecture

- In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and use it.
- Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- The 1-Tier architecture is used to develop the local application, where programmers can directly communicate with the database for quick response.



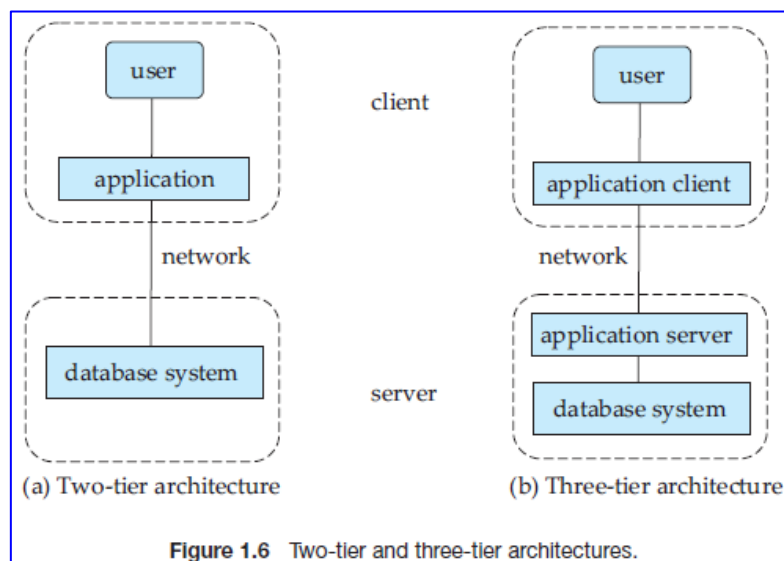
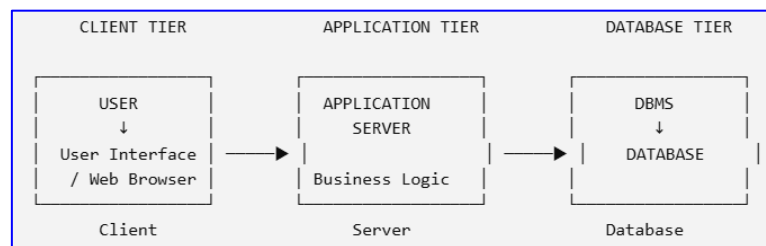
2-Tier Architecture

- The 2-Tier architecture is the same as the basic client-server. In the two-tier architecture, applications on the client end can directly communicate with the database on the server side. For this interaction, API's like: **ODBC, JDBC** are used.
- The user interfaces and application programs are run on the client side.
- The server side is responsible for providing the functionalities like: query processing and transaction management.
- To communicate with the DBMS, the client-side application establishes a connection with the server side.



3-Tier Architecture

- The 3-Tier architecture contains another layer between the client and server. In this architecture, the client can't directly communicate with the server.
- The application on the client-end interacts with an application on server which further communicates with the database system.
- End user has no idea about the existence of the database beyond the application server. The database also has no idea about any other user beyond the application.
- The 3-Tier architecture is used in case of large web application, and for applications that run on the WorldWideWeb.



Database Users and Administrators

A primary goal of a database system is to retrieve information from and store new information into the database. People who work with a database can be categorized as database users or database administrators.

Database Users:

Users are differentiated by the way they expect to interact with the system:

A. Application programmers:

- Application programmers are computer professionals who write application programs that interact with the database.
- They use various tools to develop user interfaces through which end users can access and manipulate data.
- They often work with programming languages and APIs like ODBC or JDBC to connect applications with the database.

Example:

A software developer creating a banking application that allows customers to view account balances, transfer money, and generate reports.

They might use Rapid Application Development (RAD) tools to design forms and reports quickly without writing full code from scratch.

B. Sophisticated users:

- Sophisticated users interact with the database without writing full application programs.
- Instead, they form their requests directly in a database query language (e.g., SQL).
- Their queries are processed by the query processor, which translates the DML statements into instructions that the storage manager can understand.
- These users generally have a good understanding of database concepts and can retrieve or manipulate data as needed.

Example:

A data analyst who writes SQL queries like:

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

C. Specialized users :

- - Specialized users are sophisticated users who write specialized database applications that do not fit into the traditional data-processing framework.
 - Among these applications are computer-aided design systems, knowledge base and expert systems, systems that store data with complex data types (for example, graphics data and audio data), and environment-modeling systems.

Example: A medical researcher working with a knowledge base or expert system to store and analyze patient data, medical images, or genetic sequences.

- Naïve users :

- Naive users are unsophisticated users who interact with the system by invoking one of the application programs that have been written previously.
- For **example**, a bank teller who needs to transfer \$50 from account A to account B invokes a program called **transfer**. This program asks the teller for the amount of money to be transferred, the account from which the money is to be transferred, and the account to which the money is to be transferred.

Query Processor:

The query processor will accept query from user and solves it by accessing the database.

Parts of Query processor:

1. DDL interpreter

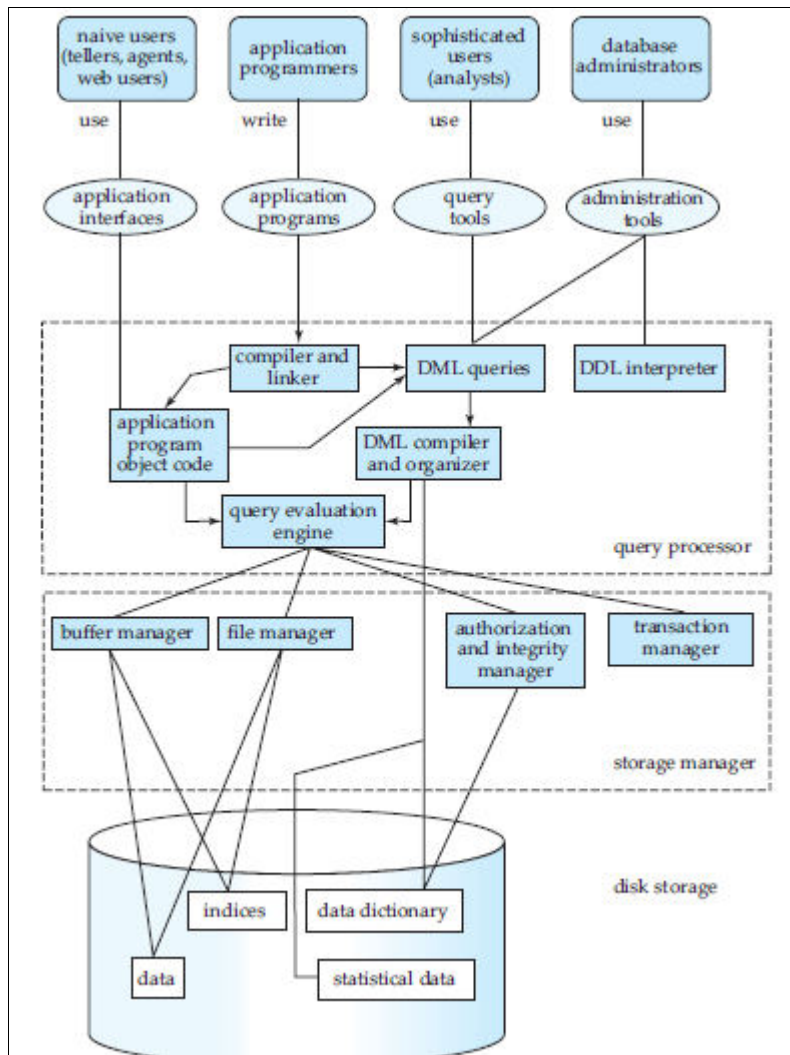
- a. This will interprets DDL statements and fetch the definitions in the data dictionary.

2. DML compiler

- a. This will translates DML statements in a query language into low level instructions that the query evaluation engine understands.
- b. A query can usually be translated into any of a number of alternative evaluation plans for same query result DML compiler will select best plan for query optimization.

3. Query evaluation engine

This engine will execute low-level instructions generated by the DML compiler on DBMS.



1.12.2 Database Administrator

One of the main reasons for using DBMSs is to have central control of the data and the programs that access those data. A person with such central control over the system is called a **database administrator (DBA)**.

The functions of a DBA include:

1. **Schema definition:** The DBA creates the original database schema by executing a set of data definition statements in the DDL.

2. **Storage structure and access-method definition.:** The DBA will select a suitable storage structure and method to access data based on the type of database.
3. **Schema and physical-organization modification:** The DBA carries out changes to the schema and physical organization to reflect the changing needs of the organization, or to alter the physical organization to improve performance.
4. **Granting of authorization for data access.** By granting different types of authorization, the database administrator can regulate which parts of the database various users can access. The authorization information is kept in a special system structure that the database system consults whenever someone attempts to access the data in the system.
5. **Routine maintenance.** Examples of the database administrator's routine maintenance activities are:
 - a. **Periodically backing up the database,** either onto tapes or onto remote servers, to prevent loss of data in case of disasters such as flooding.
 - b. **Ensuring that enough free disk space is available for normal operations and upgrading disk space** as required.
 - c. **Monitoring jobs running on the database and ensuring that performance is not degraded** by very expensive tasks submitted by some users.
 - d. **Database tuning:** DBAs tune databases to maximize system resources and improve efficiency.
 - e. **Error checking:** DBAs check for database and server errors